Chapter 6: Statistical Inference: n-gram Models over Sparse Data

Majority of slides are from
Jonathan Henke, UCB and William M. Pottenger, Lehigh University

Basic Idea:

- Examine short sequences of words
- How likely is each sequence?
- "Markov Assumption" – word is affected only by its "prior local context" (last few words)

Possible Applications:

- OCR / Voice recognition – resolve ambiguity
- Spelling correction
- Machine translation
- Confirming the author of a newly discovered work
- "Shannon game"

"Shannon Game"

- Claude E. Shannon. "Prediction and Entropy of Printed English", *Bell System Technical Journal* 30:50-64. 1951.
  - Next character
- Predict the next word, given (*n-1)* previous words
- Determine probability of different sequences by examining training corpus

Forming Equivalence Classes (Bins)

- "*n-gram*" = sequence of n words
  - bigram
  - trigram
  - four-gram
- Consider the bigrams in the vocabulary of parents of kids who play soccer…
  - 'Soccer' followed by ???
    - mom vs. game vs. ???

5

Reliability vs. Discrimination

"large green _____"

tree? mountain? frog? car? pill?

→ *more occurrences of more general trigram*

"swallowed the large green _____"

pill? broccoli?

→ *fewer occurrences of more precise 5-gram*

6

Reliability vs. Discrimination

- larger n:
  - more information about the context of the specific instance (greater discrimination)
  - E.g., noun phrases
- smaller n:
  - more instances in training data, better statistical estimates (more reliability)
  - E.g., bigram collocations

7

Selecting an *n*

Vocabulary (V) = 20,000 words

if we divide the data into too many bins, then there are a lot of parameters to estimate.

n	Number of probabilities in model
2 (bigrams)	~400,000,000
3 (trigrams)	~8,000,000,000,000
4 (4-grams)	~1.6×10^{17}

8

Statistical Estimators

- Despite lack of linguistic, sentence structure and other data, it is difficult to beat a statistical estimator based on a trigram model for predicting the next word…! Statistical estimators do work.
- So, given the observed training data …
- How do you develop a model (probability distribution) to predict future events?

9

Statistical Estimators

Example:

Corpus: five Jane Austen novels

N = 617,091 words

V = 14,585 unique words

Task: predict the next word of the trigram "inferior to _____"

from test data, *Persuasion*: "[In person, she was] inferior to *both* [sisters.]"

10

Statistical Estimators

The task of predicting the next word can be stated as attempting to estimate the probability function P:

$$P(w_n|w_1 \cdots w_{n-1}) = \frac{P(w_1 \cdots w_n)}{P(w_1 \cdots w_{n-1})}$$

- We use a classification of the previous words, the to predict the next word.
- $P(w_1 \ldots w_n)$ is frequency of a certain n-gram in the training text

11

Statistical Estimators

N	Number of training instances
B	Number of bins training instances are divided into
w_{1n}	An n-gram $w_1 \cdots w_n$ in the training text
$C(w_1 \ldots w_n)$	Frequency of n-gram $w_1 \cdots w_n$ in training text
r	Frequency of an n-gram
$f(\cdot)$	Frequency estimate of a model
N_r	Number of bins that have r training instances in them
T_r	Total count of n-grams of frequency r in further data
h	'History' of preceding words

Table 6.2 Notation for the statistical estimation chapter.

N: # of words
V: distinct words (vocabulary)
B: V^n which is the # of possible n-grams
N_r : # of n-grams that are appeared r times in training text (frequencies of frequencies)

12

Statistical Estimators

Maximum Likelihood Estimation (MLE)

Regardless of how we form equivalence classes, we will end up with bins that contain a certain number of training instances. Let us assume a trigram model where we are using the two preceding words of context to predict the next word, and let us focus in on the bin for the case where the two preceding words were comes ***across***. In a certain corpus, the authors found 10 training instances of the words comes ***across***, and of those, 8 times they were followed by ***as***, once by ***more*** and once by a. The question at this point is what probability estimates we should use for estimating the next word.

The obvious first answer (at least from a frequentist point of view) is to suggest using the *relative frequency* as a probability estimate:

$P(as) = 0.8$
$P(more) = 0.1$
$P(a) = 0.1$
$P(x) = 0.0$

$$P_{MLE}(w_1 \ldots w_n) = \frac{C(w_1 \ldots w_n)}{N}$$

$$P(w_n | w_1 \ldots w_{n-1}) = \frac{P(w_1 \ldots w_n)}{P(w_1 \ldots w_{n-1})}$$

This estimate is called the *maximum likelihood estimate* (MLE):

Statistical Estimators

While a few words are common, the vast majority of words are very uncommon and longer n-grams involving them are thus much rarer again. The MLE assigns a zero probability to unseen events, and since the probability of a long string is generally computed by multiplying the probabilities of subparts, these zeroes will propagate and give us bad (zero probability) estimates for the probability of sentences when we just happened not to see certain n-grams in the training text." With respect to the example above, the MLE is not capturing the fact that there are other words which can follow *comes across* for example *the* and *some.*

Statistical Estimators

The maximum likelihood estimate is so called because it is the choice of parameter values which gives the highest probability to the training data.

The estimate that does that is the one shown above. It does not waste any probability mass on events that are not in the training corpus,

The problem is the sparseness of our data !

In	*person*	*she*		*was*		inferior		*to*		*both*		*sisters*	
1-gram	$P(\cdot)$			$P(\cdot)$		$P(\cdot)$		$P(\cdot)$		$P(\cdot)$		$P(\cdot)$	
	the*	0.034		the	0.034	the	0.034	the	0.034	the	0.034	the	0.034
	to	0.032		to	0.032	to	0.032	**to**	*0.032*	to	*0.032*	to	0.032
	and	0.030		and	0.030	and	0.030			and	*0.030*	and	0.030
	of	0.029		of	0.029	of	0.029			of	0.029	of	0.029
8	was	0.015		was	0.015	was	0.015			was	0.015	was	0.015
13	she	0.011				she	0.011			she	0.011	she	0.01 1
284						both	0.0005			both	0.0005	both	0.0005
435						sisters	0.0003					sisters	0.0003
1701						inferior	0.00005						
Z-gram	$P(\cdot \mid person)$		$P(\cdot \mid she)$			$P(\cdot \mid was)$		$P(\cdot \mid inferior)$		$P(\cdot \mid to)$		$P(\cdot \mid both)$	
1	and	0.099	had	0.141		not	0.065	**to**	*0.2 12*	be	0.111	of	0.066
2	who	0.099	was	0.122		a	0.052			the	0.057	to	0.041
3	to	0.076				the	0.033			her	0.048	in	0.038
4	in	0.045				to	0.031			have	0.027	and	0.025
23	she	0.009								Mrs	0.006	she	0.009
41										what	0.004	sisters	0.006
293										**both**	0.0004		
∞						inferior	0						
3-gram	$P(\cdot \mid In,person)$		$P(\cdot \mid person,she)$			$P(\cdot \mid she,was)$		$P(\cdot \mid was,inf.)$		$P(\cdot \mid inferior,to)$		$P(\cdot \mid to,both)$	
	UNSEEN		did	0.5		not	*0.057*	UNSEEN		the	0.286	to	0.222
2			was	0.5		very	*0.038*			Mana	0.143	Chapter	0.111
						in	0.030			cherries	0.143	Hour	0.111
4						to	0.026			her	0.143	Twice	0.111
∞						inferior	0			both	0	sisters	0
4-gram	$P(\cdot \mid u,l,p)$		$P(\cdot \mid l,p,s)$			$P(\cdot \mid p,s,w)$		$P(\cdot \mid s,w,i)$		$P(\cdot \mid w,i,t)$		$P(\cdot \mid i,t,b)$	
	UNSEEN		UNSEEN			in	1.0	UNSEEN		UNSEEN		UNSEEN	
						inferior	0						

Statistical Estimators

•The **unigram** distribution ignores context entirely, and simply uses the overall frequency of different words.
•The **bigram** model uses the preceding word to help predict the next word. In general, this helps enormously, and gives us a much better model. In some cases the estimated probability of the word that actually comes next has gone up by about an order of magnitude (was, to, *sisters).*
•The **trigram** work brilliantly. For example, it gives us a probability estimate of 0.5 for *was* following *person she.*
But in general it is not usable. Either the preceding bigram was never seen before, and then there is no probability distribution for the following word, or a few words have been seen following that bigram but the data is so sparse that the resulting estimates are highly unreliable.

17

Statistical Estimators

Examining the table suggests an obvious strategy: use higher order n-gram models when one has seen enough data for them to be of some use, but back off to lower order n-gram models when there isn't enough data.

Need a way to give a non-zero probability estimate to words or n-grams that we don't see in training text

18

"Smoothing"

- Develop a model which decreases probability of seen events and allows the occurrence of previously unseen n-grams
- a.k.a. "Discounting methods"
- "Validation" – Smoothing methods which utilize a second batch of test data.

19

Statistical Estimators

Laplace:

$$P_{\text{Lap}}(w_1 \cdots w_n) = \frac{C(w_1 \cdots w_n) + 1}{N + B}$$

Often informally referred to as *adding one,* has the effect of giving a little bit of the probability space to unseen events.

Problem: assumes a uniform prior on events (i.e., that every n-gram was equally likely).
Dependent on the size of the vocabulary. For sparse sets of data over large vocabularies, such as n-grams, Laplace's law actually gives far too much of the probability space to unseen events.

20

Statistical Estimators

Example:

Corpus: five Jane Austen novels

N = 617,091 words

V = 14,585 unique words

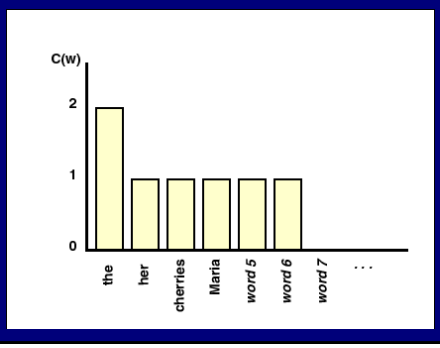
Task: predict the next word of the trigram "inferior to _____"

from test data, *Persuasion*: "[In person, she was] inferior to *both* [sisters.]"

21

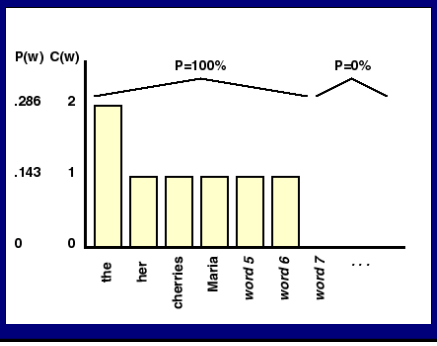
Instances in the Training Corpus:

"inferior to _____"



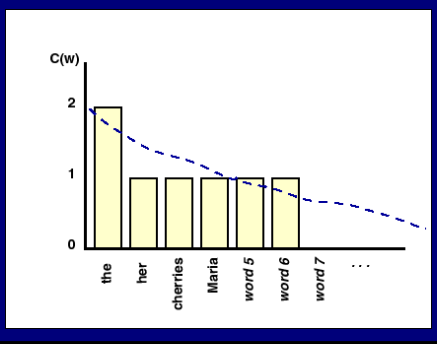
22

Maximum Likelihood Estimate:



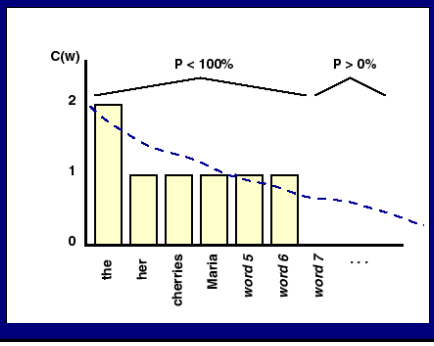
23

Actual Probability Distribution:



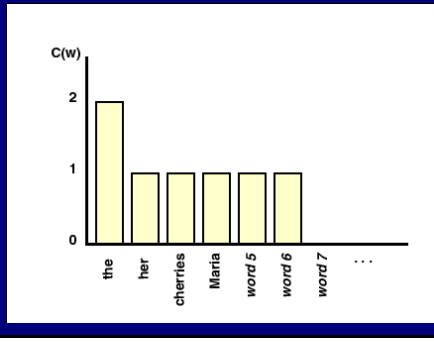
24

Actual Probability Distribution:



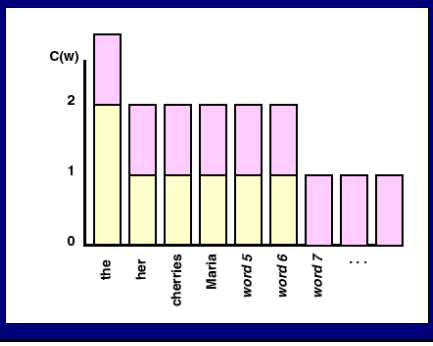
25

LaPlace's Law
(adding one)



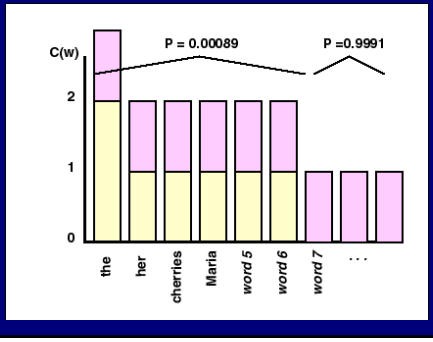
26

LaPlace's Law
(adding one)



27

LaPlace's Law



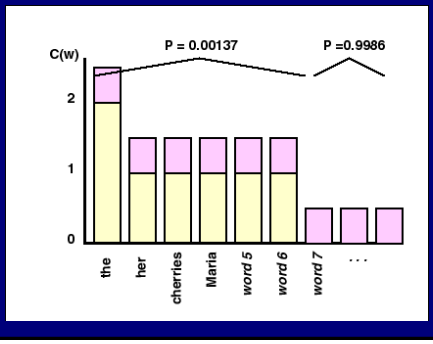
28

LaPlace

$r = f_{MLE}$	$f_{empirical}$	f_{Lap}	f_{del}	f_{GT}	N_r	t_r
0	0.000027	0.000137	0.000037	0.000027	74 671 100 000	2 019 187
	0.448	0.000274	0.396	0.446	2 018 046	903 206
2	1.25	0.000411	1.24	1.26	449 721	564 153
3	2.24	0.000548	2.23	2.24	188 933	424 015
4	3.23	0.000685	3.22	3.24	105 668	341 099
5	4.21	0.000822	4.22	4.22	68 379	287 776
6	5.23	0.000959	5.20	5.19	48 190	251951
7	6.21	0.00109	6.21	6.21	35 709	221 693
8	7.21	0.00123	7.18	7.24	27 710	199 779
9	8.26	0.00137	8.18	8.25	22 280	183 971

Table 6.4 Estimated frequencies for the AP data from Church and Gale (1991a). The first five columns show the estimated frequency calculated for a bigram that actually appeared r times in the training data according to different estimators: r is the maximum likelihood estimate, $f_{empirical}$ uses validation on the test set, f_{Lap} is the 'add one' method, f_{del} is deleted interpolation (two-way cross validation, using the training data), and f_{GT} is the Good-Turing estimate. The last two columns give the frequencies of frequencies and how often bigrams of a certain frequency occurred in further text.

Jeffreys-Perks Law



31

Lidstone's Law

$$P_{Lid}(w_1 \cdots w_n) = \frac{C(w_1 \cdots w_n) + \lambda}{N + B\lambda}$$

P = probability of specific n-gram

C = count of that n-gram in training data

N = total n-grams in training data

B = number of "bins" (possible n-grams)

λ = small positive number

M.L.E: λ = 0
LaPlace's Law: λ = 1
Jeffreys-Perks Law: λ = ½

30

Objections to Lidstone's Law

- **Need an *a priori* way to determine λ.**
- **Predicts all unseen events to be equally likely**
- **Gives probability estimates linear in the M.L.E. frequency**

32

Smoothing

- **Lidstone's Law (incl. LaPlace's Law and Jeffreys-Perks Law): modifies the observed *counts***
- **Other methods: modify *probabilities.***

33

Held-Out Estimator

- **How much of the probability distribution should be "held out" to allow for previously unseen events?**
- **Validate by holding out part of the training data.**
- **How often do events unseen in training data occur in validation data?**
  (e.g., to choose λ for Lidstone model)

34

Held-Out Estimator

- We can take further text (assumed to be from the same source) and see how often bigrams that appeared ***r*** times in the training text tend to turn up in the further text. The realization of this idea is the held out ***estimator*** of Jelinek and Mercer(1985).

35

Held-Out Estimator

$C_1(w_1 \cdots w_n)$ = frequency of $w_1 \cdots w_n$ in training data
$C_2(w_1 \ldots w_n)$ = frequency of $w_1 \cdots w_n$ in held out data

$$T_r = \sum_{\{w_1 \cdots w_n : C_1(w_1 \cdots w_n) = r\}} C_2(w_1 \ldots w_n)$$

$$P_{\text{ho}}(w_1 \cdots w_n) = \frac{T_r}{N_r N} \quad \text{where } C(w_1 \cdots w_n) = r$$

$$P_{ho}(w_1 \cdots w_n) = \frac{\sum_{\{w_1 \cdots w_n\}} C_r(w_1 \cdots w_n)}{N_r N}$$

N_r = number of *n*-grams with frequency r in training set: r = $C(w_1 \ldots w_n)$

T_r = total times all *n*-grams (that appeared r times in training set) appear in held out data

T_r / N_r = average frequency of held out *n*-grams

36

Testing Models

- Hold out ~ 5 – 10% for testing
  - A cardinal sin in Statistical NLP is to test on your training data!
- Hold out ~ 10% for validation (smoothing)
- For testing: useful to test on multiple sets of data, report variance of results.
  - Are results (good or bad) just the result of chance?

37

Cross-Validation
(a.k.a. deleted estimation)

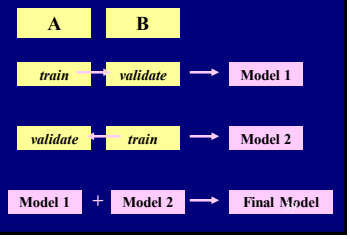
- Use data for both training and validation

Divide test data into 2 parts

(1) Train on A, validate on B

(2) Train on B, validate on A

Combine two models



Testing

	System 1	System 2
scores	71, 61, 55, 60, 68, 49, 42, 72, 76, 55, 64	42, 55, 75, 45, 54, 51 55, 36, 58, 55, 67
total	609	526
n	11	11
mean $\bar{x}_i$	55.4	47.8
$s_i^2 = \sum(x_{ij} - \bar{x}_i)^2$	1,375.4	1,228.8
df	10	10

Pooled $s^2 = \frac{1375.4+1228.8}{10+10} \approx 130.2$

$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{2s^2}{n}}} = \frac{55.4-47.8}{\sqrt{\frac{2 \cdot 130.2}{11}}} \approx 1.56$

Table 6.6 **Using** the t test for comparing the performance of two systems. Since we calculate the mean for each data set, the denominator in the calculation of variance and the number of degrees of freedom is (11 – 1) + (11 – 1) = 20. The data do not provide clear support for the superiority of system 1. Despite the clear difference in mean scores, the sample variance is too high to draw any definitive conclusions.

Cross-Validation

Two estimates:

$$P_{ho} = \frac{T_r^{01}}{N_r^0 N} \qquad P_{ho} = \frac{T_r^{10}}{N_r^1 N}$$

N_r^a = number of n-grams occurring r times in a-th part of training set

T_r^{ab} = total number of those found in b-th part

Combined estimate:

$$P_{ho} = \frac{T_r^{01} + T_r^{10}}{N(N_r^0 + N_r^1)}$$

(weighted average in proportion to the number of words in each half)

40

Cross-Validation
(a.k.a. deleted estimation)

- On large training corpora, doing deleted estimation on the training data works better than doing held-out estimation using just the training data
- Table 6.4 shows that it produces results that are quite close to the empirical gold standard.
- But it is still some way off for low frequency events.
- It overestimates the expected frequency of unseen objects
- It underestimating the expected frequency of objects that were seen once in the training data.

41

Good-Turing Estimator

- Good (1953) attributes to Turing a method for determining frequency or probability estimates of items
- Assumption is that their distribution is binomial.
- Suitable for large numbers of observations of data drawn from a large vocabulary
- Works well for n-grams (despite the fact that words and n-grams do not have a binomial distribution)

42

Good-Turing Estimator

$$r^* = (r+1)\frac{E(N_{r+1})}{E(N_r)} \qquad P_{GT} = r^*/N$$

r^* = "adjusted frequency"

N_r = number of *n-gram*-types which occur *r* times

$E(N_r)$ = "expected value"

$E(N_{r+1}) < E(N_r)$

(See textbook for example application)

43

Good-Turing Estimator

- E denotes the expectation of a random variable
- The total probability mass reserved for unseen objects is $E(N_1)/N$
- Using our empirical estimates, we can hope to substitute the observed Nr for E(Nr)
- However, we cannot do this uniformly!
  - The most frequent n-gram would be estimated to have probability zero, since the number of n-grams with frequency one greater than it is zero!
- Solution 1: Use Good-Turing reestimation only for frequencies ***r<k*** for some constant ***k (e.g., 10).***
  - Low frequency words are numerous, so substitution of the observed frequency of frequencies for the expectation is quite accurate
  - The MLE estimates of high frequency words will also be quite accurate and so one doesn't need to discount them.

44

Good-Turing Estimator

- Solution 2: fit some function S through the observed values of (Y, N_r)
- se the smoothed values S(r) for the expectation
  - leads to a family of possibilities depending on exactly which method of curve fitting is employed
- The probability mass N_1/N given to unseen items can either be divided among them uniformly, or by somewhat more sophisticated method
  - (see under Combining Estimators, below).

Good-Turing Estimator: If $C(w_1 \dots w_n) = r > 0$,

$$P_{GT}(w_1 \cdots w_n) = \frac{r^*}{N} \quad \text{where } r^* = \frac{(r+1)S(r+1)}{S(r)}$$

If $C(w_1 \dots w_n) = 0$,

$$P_{GT}(w_1 \cdots w_n) = \frac{1 - \sum_{r=1}^{\infty} N_r \frac{r^*}{N}}{N_0} \approx \frac{N_1}{N_0 N}$$

Good-Turing Estimator

1. Calculate the frequencies of different frequencies (also known as count-counts).

Bigrams				Trigrams			
r	N_r	r	N_r	r	N_r	r	N_r
1	138741	28	90	1	404211	28	35
2	25413	29	120	2	32514	29	32
3	10531	30	86	3	10056	30	25
4	5997	31	98	4	4780	31	18
5	3565	32	99	5	2491	32	19
6	2486		...	6	1571		..
7	1754	1264	1	7	1088	189	1
8	1342	1366	1	8	749	202	1
9	1106	1917	1	9	582	214	1
10	896	2233	1	10	432	366	1
	...	2507	1		...	378	1

Table 6.7 Extracts from the frequencies of frequencies distribution for bigrams and trigrams in the Austen corpus.

Good-Turing Estimator

$r = f_{MLE}$	$f_{empirical}$	f_{Lap}	f_{del}	f_{GT}	N_r	t_r
0	0.000027	0.000137	0.000037	0.000027	74 671 100 000	2 019 187
	0.448	0.000274	0.396	0.446	2 018 046	903 206
2	1.25	0.000411	1.24	1.26	449 721	564 153
3	2.24	0.000548	2.23	2.24	188 933	424 015
4	3.23	0.000685	3.22	3.24	105 668	341 099
5	4.21	0.000822	4.22	4.22	68 379	287 776
6	5.23	0.000959	5.20	5.19	48 190	251951
-	6.21	0.00109	6.21	6.21	35 709	221 693
8	7.21	0.00123	7.18	7.24	27 710	199 779
9	8.26	0.00137	8.18	8.25	22 280	183 971

Table 6.4 Estimated frequencies for the AP data from Church and Gale (1991a). The first five columns show the estimated frequency calculated for a bigram that actually appeared r times in the training data according to different estimators: r is the maximum likelihood estimate, $f_{empirical}$ uses validation on the test set, f_{Lap} is the 'add one' method, f_{del} is deleted interpolation (two-way cross validation, using the training data), and f_{GT} is the Good-Turing estimate. The last two columns give the frequencies of frequencies and how often bigrams of a certain frequency occurred in further text.

Simple Good-Turing can give exceedingly good estimators !

Good-Turing Estimator

For the bigrams, the mass reserved for unseen bigrams, N_1/N = 138741/617091 = 0.2248. The space of bigrams is the vocabulary squared, and we saw 199,252 bigrams, so using uniform estimates, the probability estimate for each unseen bigram is: $0.2248/(14585^2 - 199252) = 1.058 \times 10^{-9}$. If we now wish to work out conditional probability estimates for a bigram model by using Good-Turing estimates for bigram probability estimates, and MLE estimates directly for unigrams, then we begin as follows:

$$P(she|person) = \frac{f_{GT}(person\ she)}{C(person)} = \frac{1.228}{223} = 0.0055$$

Good-Turing Es

Still suffers from assuming a uniform distribution over unseen bigrams!

$P(she\|person)$	0.0055
$P(was\|she)$	0.1217
$P(inferior\|was)$	6.9×10^{-8}
$P(to\|inferior)$	0.1806
$P(both\|to)$	0.0003956
$P(sisters\|both)$	0.003874

Table 6.9 Good-Turing bigram frequency estimates for the clat sion.

r	r^*	$P_{GT}(\cdot)$
0	0.0007	1.058×10^{-9}
1	0.3663	5.982×10^{-7}
2	1.228	2.004×10^{-6}
3	2.122	3.465×10^{-6}
4	3.058	4.993×10^{-6}
5	4.015	6.555×10^{-6}
6	4.984	8.138×10^{-6}
7	5.96	9.733×10^{-6}
8	6.942	1.134×10^{-5}
9	7.928	1.294×10^{-5}
10	8.916	1.456×10^{-5}
...		
28	26.84	4.383×10^{-5}
29	27.84	4.546×10^{-5}
30	28.84	4.709×10^{-5}
31	29.84	4.872×10^{-5}
32	30.84	5.035×10^{-5}
...		
1264	1263	0.002062
1366	1365	0.002228
1917	1916	0.003128
2233	2232	0.003644
2507	2506	0.004092

Absolute/Linear Discounting

Ney and Essen (1993) and Ney et al. (1994) propose two discounting models: in the absolute discounting model, all non-zero MLE frequencies are discounted by a small constant amount δ and the frequency so gained is uniformly distributed over unseen events:

Absolute discounting: If $C(w_1 \ldots w_n) = r$,

$$P_{\text{abs}}(w_1 \cdots w_n) = \begin{cases} (r-\delta)/N & \text{if } r > 0 \\ \frac{(B-N_0)\delta}{N_0 N} & \text{otherwise} \end{cases}$$

(Recall that B is the number of bins.) In the linear discounting method, the non-zero MLE frequencies are scaled by a constant slightly less than one, and the remaining probability mass is again distributed across novel events:

Linear discounting: If $C(w_1 \ldots w_n) = r$,

$$P(w_1 \cdots w_n) = \begin{cases} (1-\alpha)r/N & \text{if } r > 0 \\ \alpha/N_0 & \text{otherwise} \end{cases}$$

Other Discounting Methods

- ***Absolute discounting:*** **Decrease probability of each observed n-gram by subtracting a small constant and distribute the resulting probability mass uniformly over unseen events**
- ***Linear discounting:*** **Decrease probability of each observed n-gram by multiplying by a constant slightly less than one and distribute the resulting probability mass uniformly over unseen events**

Combining Estimators

(Sometimes a trigram model is best, sometimes a bigram model is best, and sometimes a unigram model is best.)

- **How can you develop a model to utilize different length *n-grams* as appropriate?**

Simple Linear Interpolation

- So far the methods we have considered raw frequency of an n-gram and have tried to produce the best estimate of its probability in future text from that.
- But rather than giving the same estimate for all n-grams that never appeared or appeared only rarely, we could hope to produce better estimates by looking at the frequency of the (n-l)-grams found in the n-gram.
  - If these (n-l)-grams are themselves rare, then we give a low estimate to the n-gram.
  - If the (n-1)-grams are of moderate frequency, then we give a higher probability estimate for the n-gram.

Simple Linear Interpolation

(a.k.a., finite mixture models; a.k.a., deleted interpolation)

$$P_{li}(w_n \mid w_{n-2}, w_{n-1}) =$$

$$\lambda_1 P_1(w_n) + \lambda_2 P_2(w_n \mid w_{n-1}) + \lambda_1 P_3(w_n \mid w_{n-2}, w_{n-1})$$

- **Weighted average of unigram, bigram, and trigram probabilities**
- **Suffers less from data sparseness present in some trigram-only models**

Simple Linear Interpolation

- How to determine weights?
  - by hand…
  - by a simple application of the Expectation Maximization (EM) algorithm (section 9.2.1)

Katz's Backing-Off

- Use *n-gram* probability when enough training data
  - (when adjusted count > k; k usu. = 0 or 1)
- If not, "back-off" to the *(n-1)-gram* probability
- (Repeat as needed)
- Back-off may be used to smooth or to combine information sources

$$P_{\text{bo}}(w_i \mid w_{i-n+1} \cdots w_{i-1}) = \begin{cases} (1 - d_{w_{i-n+1} \cdots w_{i-1}}) \frac{C(w_{i-n+1} \cdots w_i)}{C(w_{i-n+1} \cdots w_{i-1})} \\ \quad \text{if } C(w_{i-n+1} \cdots w_i) > k \\ \alpha_{w_{i-n+1} \cdots w_{i-1}} P_{\text{bo}}(w_i \mid w_{i-n+2} \cdots w_{i-1}) \\ \quad \text{otherwise} \end{cases}$$

- We can start with four-gram and end up using unigram freq.

Katz's Backing-Off

$$P_{bo}(w_i|w_{i-n+1}\ldots w_{i-1}) = \begin{cases} (1 - d_{w_{i-n+1}\cdots w_{i-1}}) \frac{C(w_{i-n+1}\cdots w_i)}{C(w_{i-n+1}\cdots w_{i-1})} \\ \quad \text{if } C(w_{i-n+1}\cdots w_i) > k \\ \alpha_{w_{i-n+1}\cdots w_{i-1}} P_{bo}(w_i|w_{i-n+2}\cdots w_{i-1}) \\ \quad \text{otherwise} \end{cases}$$

- If the n-gram has appeared more than ***k*** times is (k normally set to 0 or 1) then an n-gram estimate is used, as in the first line.
- The MLE estimates need to be discounted in some manner, or else there would be no probability mass to distribute to the lower order models (represented as d).
  - use the Good-Turing estimates discussed above
- Back-off probability has to be multiplied by a normalizing factor **α** so that only the probability mass left over in the discounting process is distributed among n-grams

57

Problems with Backing-Off

- **If bigram $w_1\ w_2$ is common**
- **but trigram $w_1\ w_2\ w_3$ is unseen**
- **may be a *meaningful* gap, rather than a gap due to chance and scarce data**
  - **i.e., a "grammatical null"**
- **May not want to back-off to lower-order probability**

59

Katz's Backing-Off

$$P_{bo}(w_i|w_{i-n+1}\cdots w_{i-1}) = \begin{cases} d_{w_{i-n+1}\cdots w_{i-1}} \frac{C(w_{i-n+1}\ldots w_i)}{C(w_{i-n+1}\cdots w_{i-1})} \text{ if } C(w_{i-n+1}\cdots w_i) > k \\ \alpha_{w_{i-n+1}\cdots w_{i-1}} P_{bo}(w_i|w_{i-n+2}\cdots w_{i-1}) \text{ otherwise} \end{cases}$$

- **How to determine parameters:**
  - k (usually 0 or 1)
  - d (amount of discounting found by Good-Turing estimation)
  - α: back-off weight, determines how much this (n-1)-gram should use from the left-over probability mass for unseen n-grams

Applications

- http://en.wikipedia.org/wiki/N-gram

63